ポートフォリオ / 2026

予約アプリを1人で作って1年運用しました

谷口昂太郎

作ったもの

オンライン塾の予約アプリ。LINE の中で開く（LIFF）。

2ページ目（日時）を見る

- 学年 → 教科 → 単元 → 授業内容
- 開くと、その下の項目が出る
- 戻るボタンが要らない



30〜40
予約 / 月（10人から）

10分
1件あたり（2日から）

1年
開発から運用まで

ユーザーはこの順で動く

1. 公式LINE から、LIFF を開く
2. LINE ログイン（初回だけ）
3. 学年を選ぶ
4. 教科を選ぶ
5. 単元と授業内容を選ぶ
6. カレンダーで日時を選ぶ
7. 確認して、送信する
8. LINE に日時と URL が届く

予約が通るまで



同時予約だけ Firebase で止める。記録は Spreadsheet に残す。

1件の予約で何が起きるか



要件 → 制約 → 設計

要件

- 授業情報は Spreadsheet のまま
- とにかく早く
- LINE と Slack に通知

制約

- 同時実行を守れない
- Ticket は1件ずつ
- 公開前は Ticket 不可

設計

- Firebase で直列化
- PATCH は取得→追加→上書き
- 公開を先に回す

正のデータを人が触る

- アプリと人が同じ Spreadsheet に書く
- 列のセルが置き換わり、今日が出なくなる
- 壊れたことに、あとから気づく

複雑さを構造で解く

課題	手	結果
状態が8つ	XState で遷移を表に	表に無い遷移が起きない
密結合	責務で分ける	変更が1か所で済む
外部 API	境界の外に置く	仕様変更に耐える

自分で提案して変えた

- アコーディオン：6画面を1画面に（自分のアイデア）
- リアーキテクチャ：依存の向きを、自分で手を動かして分けた
- 結果：変更が怖くなくなった

1年運用して分かったこと

- 要件が設計を決める
- 品質は後追いになる
- 1人で抱えると、詰まる

御社でやりたいこと

- 設計レビューを「受ける側」から「出す側」へ
- フロントの品質（テストと状態設計）を上げる
- 1人で抱えず、チームで速くする

付録：技術スタックと年表

領域	使ったもの
型	TypeScript / Zod（Branded Type）/ @t3-oss/env-nextjs
画面	Next.js / React / Jotai / XState / Tailwind / Day.js
基盤	LIFF（LINE ログイン）/ Firebase / Vercel
データ	google-spreadsheet / google-auth-library（Service Account）
外部	Zoom Events API（Ticket / Session）
品質	Biome / Vitest / Knip / React Doctor

フォームは最初 React Hook Form。後に外して、React と Jotai に寄せた。